dlaczego robimy zmianę

refactor-opportunities

Notatka kontekstowa dla laika — od mapy projektu do decyzji „co naprawiamy”

Repozytorium: apache/maven (Apache Maven 4)
Zmiana: context/changes/refactor-opportunities
Status: implemented (K2 zrealizowane) · branch fix/test-implementation · odświeżono 2026-10-14
Źródło prawdy: repo-map → model-builder-flow-analysis → research → plan K2 → plan-review → implementacja (3 fazy)

W jednym zdaniu

Nie przebudowaliśmy całego Mavena. Zmapowaliśmy, gdzie boli; namierzyliśmy dług w budowaniu modelu POM; wybraliśmy najtańsze, krytyczne zabezpieczenie — i je zrobiliśmy: dokumentacja + testy chronią celową różnicę Maven 3↔4 (FATAL vs WARNING przy złym relativePath / personality). Semantyka produkcyjna bez zmian (pełne K2, status implemented).

Spis treści (ścieżka rozumowania) — kliknij, aby przejść

1. Czym jest Maven — bez żargonu
2. Skąd się wzięła mapa projektu (repo-map)
3. Co mapa mówi o ryzyku
4. Poprzedni krok: analiza przepływu ModelBuildera
5. Czym jest zmiana refactor-opportunities
6. Trzej kandydaci (K1, K2, K3) — w prostych słowach
7. Dlaczego wybraliśmy właśnie K2
8. Co konkretnie zrobiliśmy (i czego świadomie nie)
9. Co zaimplementowano — szczegóły kodu wg faz
10. Jak to się ma do całego M4 / „realizing change”
11. Podsumowanie dla kogoś spoza zespołu
12. Historia zmian

Ta notatka idzie od szerokiego obrazu do wąskiej decyzji, a potem do tego, co faktycznie weszło w kod. Spis powyżej jest klikalny (oraz panel zakładek w czytniku PDF). Jeśli czytasz tylko jedną stronę poza okładką — sekcja 9 (implementacja).

1. Czym jest Maven — bez żargonu

Wyobraź sobie, że budujesz dom z tysiąca elementów od różnych dostawców. Ktoś musi przeczytać instrukcję („jakie ściany, jakie okna, w jakiej kolejności"), ściągnąć brakujące części i złożyć wszystko w działający budynek. Apache Maven robi coś podobnego dla projektów Java: czyta plik konfiguracyjny zwany POM (Project Object Model — opis projektu), rozwiązuje zależności i uruchamia kroki budowania (kompilacja, testy, pakowanie).

Wersja, nad którą pracuje społeczność w tym repozytorium, to przede wszystkim Maven 4 — duża modernizacja względem Mavena 3. Stary i nowy świat muszą jeszcze przez jakiś czas współistnieć: stare pluginy, stare zachowania i nowe, ostrzejsze reguły. To napięcie (most M3↔M4) jest kluczowe dla zrozumienia naszej zmiany.

2. Skąd się wzięła mapa projektu (repo-map)

Zanim ktokolwiek zaczął „refaktorować", powstała mapa onboardingowa: context/map/repo-map.md. Jej cel nie brzmi jak „naprawmy kod", tylko: gdzie żyje system, co boli, od czego czytać, kogo zapytać.

Mapa powstała z syntezy trzech artefaktów (historia gita, struktura zależności, mapa ludzi) w oknie ~12 miesięcy (2025-10 → 2026-10). Ważne ograniczenie: to nie jest pełne przeczytanie każdej linii kodu — to radar ryzyka i aktywności, żeby nie marnować energii na fałszywe centra uwagi (np. gęste poprawki w testach integracyjnych albo Dependabot).

Jak Maven jest ułożony (warstwy)

Mapa pokazuje przewidywalny układ warstw — jak piętra budynku, gdzie niższe nie powinny zależeć od wyższych:

Warstwa	Rola w prostych słowach
cli / embedder	Cienkie wejście — tu użytkownik odpala mvn
core	Runtime: kolejkowanie celów, projekty, pluginy
impl	Budowanie modelu POM + implementacje usług
api/*	Publiczny kontrakt Mavena 4 (to, czego trzyma się ekosystem)
compat/* + support	Most do Mavena 3 i wspólny I/O modelu
its/	Lustro regresji — dużo ruchu, ale nie „serce produktu"

Kierunek zależności: api ← impl ← core ← cli, równolegle most compat. Praca (po odfiltrowaniu botów) skupia się w kolejności: impl → core → ITS → cli. Boli przede wszystkim: edge case'y modelu (profile, parent, CI-friendly wersje), splątany lifecycle w core oraz każdy ruch w publicznym API.

3. Co mapa mówi o ryzyku

Mapa wyróżnia pięć stref ryzyka. Dla naszej historii najważniejsze są dwie pierwsze i most kompatybilności:

Strefa 1 — Budowanie modelu POM

To #1 aktywność i wysoki odsetek poprawek. Kod żyje głównie w impl/maven-impl (hub: DefaultModelBuilder), ze stykiem do legacy compat/maven-model-builder (~88% zmian w starym builderze idzie razem z impl). Tu rozstrzygane są profile, parent POM-y i przyjazne CI wersje w stylu ${revision}.

Strefa 2 — Lifecycle, reactor, pluginy

W impl/maven-core siedzi orkiestracja budowania wielu projektów naraz. „Mała” lokalna zmiana często tyka kilku pakietów naraz (cykle wewnętrzne).

Strefa 4 — Most M3↔M4

Świadomie utrzymywane mosty w compat/* i wspólnym I/O. Jedna zmiana w czytaniu/pisaniu modelu uderza w nowy i stary świat naraz. To nie jest „brud do wycięcia jutro” — to load-bearing kompromis podczas migracji ekosystemu.

Wniosek z mapy (dla laika)

Jeśli chcesz bezpiecznie poprawiać Maven 4, nie zaczynaj od najgłośniejszych plików w gicie. Zaczynaj od ścieżki: jak z pliku POM powstaje „model projektu”, i od tego, jak nowy kod dogaduje się ze starym mostem kompatybilności.

4. Poprzedni krok: analiza przepływu ModelBuildera

Zmiana model-builder-flow-analysis była Deep Focusem na strefę 1 z mapy. Jej intencja (z change.md): prześledzić przepływ budowania modelu POM — od odczytu pliku do przejścia model → project — zaczynając od hubu DefaultModelBuilder.

Uwaga o statusach: frontmatter change.md prioru nadal mówi preparing, ale istnieje kompletny research.md (status: complete, 2026-10-13) z trace'em e2e, długiem technicznym i lukami testowymi. To właśnie ten raport jest wejściem do refactor-opportunities.

Analiza celowo nie zamykała pytania „co naprawiamy". Udokumentowała dług: ogromny hub, bliźniaczy kod M3/M4, luki w testach, kruche sprzężenia. Zostawiła otwarte: które z tych problemów warto naprawić, w jakim kształcie i w jakiej kolejności.

Kluczowe ustalenie przepływu (dla orientacji): na hot path reactora idzie CLI → DefaultMaven → collection → project.DefaultProjectBuilder → api.ModelBuilder (impl) → materializacja MavenProject. Twin compat nie buduje modelu reactora M4 — żyje dla legacy / resolver-provider.

Bez tego kroku „refactor-opportunities" byłoby strzelaniem w ciemno. Z nim mamy listę problemów z dowodami w kodzie i historii commitów.

5. Czym jest zmiana refactor-opportunities

To kolejny etap tej samej historii — nie nowy produkt, tylko uporządkowanie długu w okazje refaktoru i decyzja, którą okazję realizujemy. Pracujemy w apache/maven (Java).

Etapy (jak w change.md)

- Eksploracja — badamy każdy zapisany problem; zero refaktoru, zero decyzji produktowej; wynik: research.md z rankingiem.
- Decyzja i plan — wybieramy opportunity i piszemy plan implementacji (plan.md + brief).
- Implementacja — według przyjętego planu; Progress faz 1–3 domknięty; status implemented.

Status dziś: implemented — research kompletny, plan SOUND, pełne K2 zrealizowane na branchu fix/test-implementation (commity p1/p2/p3 + epilogue). Szczegóły kodu: sekcja 9.

6. Trzej kandydaci (K1, K2, K3) — w prostych słowach

Research wyłonił trzech kandydatów strukturalnych. Reszta problemów (Jacoco %, luki mapy, unknown SPI) to koszt / osłony, nie „refaktor do zrobienia teraz".

K1 — „Kuchnia, w której wszystko stoi na jednym stole"

Klasa DefaultModelBuilder w impl ma ~3451 linii i ~19 współpracowników w konstruktorze. To świadomy orchestrator pipeline'u (tak miał być), ale rozmiar pliku urósł akumulacyjnie. Docelowo: wydzielić etapy do mniejszych typów w tym samym pakiecie (bez nowego publicznego API). Wartościowe, ale droższe i ryzykowne bez dobrych testów charakterystyki.

K2 — „Bliźnięta, które mają się różnić — i ktoś może to «naprawić»"

Istnieją dwa DefaultModelBuilder: nowy (impl / M4, ~3451 LOC) i stary (compat / M3, ~1465 LOC, @Deprecated since 4.0.0). Nie są spięte kompilatorem — semantyka trzeba uzgadniać ręcznie. Przy konkretnym błędzie (niespójny parent relativePath vs groupId/artifactId) M4 domyślnie krzyczy FATAL, a compat zostaje przy WARNING. Dodatkowo flaga maven.maven3Personality (w skrócie: „osobowość M3") celowo łagodzi zachowanie M4 do stylu M3. To nie bug — to kontrakt migracyjny. Brakuje jednak testów,

które chronią przed fałszywym „fix twin” (wyrównaniem severity „bo wygląda niespójnie”).

K3 — „Piętro nad kuchnią: składanie projektu w reactorze”

DefaultProjectBuilder w core (~1420 LOC) materializuje MavenProject na modelu. Warstwa jest świadoma od 2009 — nie należy wciągać jej z powrotem do ModelBuildera. Dług jest w gęstości orkiestracji i cienkich testach (tylko 3 @Test wokół extractProjectId), nie w samym istnieniu warstwy. Niższy priorytet, dopóki nie ma characterization ścieżki reactora.

Ranking z researchu (propozycja pod planowanie — nie decyzja produktu)

#	Opportunity	Dlaczego w tej kolejności
1	K2 — guardy twin / personality	Najniższy koszt zmiany, wysoki koszt błędu (fałszywy fix kontraktu)
2	K1 — extract stage’ów hubu	Największy lokalny dług; najpierw guardy z K2
3	K3 — extract BuildSession	Świadoma warstwa; najpierw characterization

Odrzucone jako „refaktor teraz”: unifikacja severity FATAL↔WARNING (to redesign kontraktu domeny), same testy/Jacoco jako kandydat, korekty mapy, scalenie ProjectBuildera z ModelBuilderem, nowe publiczne SPI „żeby rozbić hub”, mvnup / concurrent lifecycle.

7. Dlaczego wybraliśmy właśnie K2

Plan (po wywiadzie / decyzji) bierze wyłącznie pełne K2 — nie K1 i nie K3 w tej zmianie. Powody, w języku codziennym:

1) Najpierw zabezpieczamy sens, potem ruszamy meble

Zanim ktoś zacznie rozbijać 3450-liniowy hub (K1), musi być jasne, które zachowania są celowe. Bez guardów refaktor może „wyrównać" FATAL i WARNING i zepsuć kontrakt M4/M3 — wyglądając jak poprawka spójności.

2) Najtańszy blast radius

Ścieżka guard: dokumentacja + testy (+ ewentualnie 1–3 linie komentarza w kodzie). Zero zmiany semantyki produkcyjnej. Unifikacja severity to już redesign kontraktu domeny — poza tą zmianą (osobna analiza modelowania domeny / DDD, nie refaktor struktury).

3) Historia gita mówi „guard, nie rebuild"

Commity wokół MNG-8294, maven.maven3Personality i MNG-5146 pokazują świadomą eskalację ostrzeżeń w M4 oraz flagę wstecznej osobowości. Research: to świadome ograniczenie, nie przypadek do „posprzątania".

4) Luka characterization jest konkretna i mierzalna

Mamy unit na FATAL + hint (shade-style), mamy compat WARNING (MNG-5146), brakuje unit demotion przy personality i jasnej pary impl↔compat na tym samym układzie POM (WRONG_PARENT). To dziura, którą da się domknąć w 1–2 sesjach.

> Analogia
>
> Masz dwa regulatory prędkości: „nowy, surowy" i „stary, łagodny". Różnica jest celowa (bezpieczeństwo vs kompatybilność). K2 nie łączy ich w jeden. K2 wiesza tabliczkę i zakłada test, żeby ktoś w dobrej wierze nie „naprawił niespójności" i nie usunął różnicy.

8. Co konkretnie zrobiliśmy (i czego świadomie nie)

Zakres planu — trzy fazy (zrealizowane)

Faza	Dostarcza	Uwaga
1. Contract note	Rozszerzenie istniejącego komentarza w kodzie + notatka kontraktu w folderze zmiany	Tylko dokumentacja intencji (nie drugi równoległy komentarz)
2. Demotion unit	Test na layoutcie WRONG_PARENT: z personality → WARNING; bez → FATAL	SoT = MNG-5146; System property + recreate sesji ApiRunner
3. Paired gate	Review pary impl↔compat; zero nowego kodu	Potwierdzenie twin, nie kopiowanie fixture / shared test-jar

Ważne detale z planu/briefu: po demotion mismatch ścieżka może nadal kończyć się FATAL „Non-resolvable parent" (bez resolvera) — asercja dotyczy severity mismatchu, nie „udanego builda". Anti-pattern: request.userProperties(...) pod ApiRunner nie zasila configProperties. ITS MNG-8294 = opcjonalny smoke, nie gate demotion.

Świadomie poza zakresem

- Unifikacja FATAL↔WARNING między twinami (to byłby redesign kontraktu domeny)
- Extracty strukturalne K1 / K3
- Nowe SPI, migracja deprecated ProjectBuilder

- Jacoco jako cel; badanie ClassRealm / zewnętrznych ModelTransformer
- Korekty samego pliku repo-map.md

Kryterium sukcesu: kontrakt zapisany; testy chronią FATAL (default) i WARNING (personality) po stronie impl na WRONG_PARENT; compat WARNING zostaje; semantyka produktu bez zmian.

Review planu

Werdykt: SOUND (po triage; było REVISE). Grounding 5/5 ścieżek i symboli. Naprawione ustalenia z review:

- F1 — do Phase 2 Success Criteria / Progress dopisano asercję FATAL mismatch na WRONG_PARENT bez personality (obok demotion WARNING), żeby kanoniczna para nie została dziurawa.
- F2 — Phase 1 rozszerza istniejący komentarz przy warn=, zamiast dokładać drugi blok.

9. Co zaimplementowano — szczegóły kodu wg faz

Poniżej: co faktycznie weszło w repozytorium. Semantyka produkcyjna (warunek warn = defaultPath || maven3Mode) nie została zmieniona — zmieniono dokumentację, testy characterization i status zmiany.

Słownik przypominający: FATAL = błąd krytyczny; WARNING = ostrzeżenie; GA = groupId:artifactId; relativePath = ścieżka do POM rodzica; personality = flaga maven.maven3Personality; twin = para impl (M4) ↔ compat (M3).

Faza 1 — Contract note (commit 9d643a181c)

Cel: utrwalić, że rozjazd FATAL (impl) vs WARNING (compat) jest celowy — nie wolno unifikować severity.

Plik produkcyjny: impl/maven-impl/.../DefaultModelBuilder.java, metoda mismatchRelativePathAndGA. Logika bez zmian; dodano jedną linię komentarza do istniejącego bloku przy warn = …:

DefaultModelBuilder.java — dopisany komentarz kontraktu (diff +)

```
// WARNING when: Maven probed the default path (user didn't set anything),
//               OR maven3Personality is active (preserve historical lenient behaviour).
// FATAL otherwise: explicit <relativePath> pointing at the wrong artifact is a config error.
// Compat twin always reports WARNING for the same mismatch; do not unify severity across twins
.
boolean warn = defaultPath || maven3Mode;
add(warn ? Severity.WARNING : Severity.FATAL, Version.BASE, message, parent.getLocation(""));
```

Nowy plik: context/changes/refactor-opportunities/contract-twin-relativepath.md — macierz severity (impl default FATAL / personality WARNING / compat WARNING) + kotwice do Constants/Features/testów. Frontmatter change.md przeszedł wtedy w implementing.

Faza 2 — Demotion unit (commit ebcd0c331b)

Cel: ten sam layout co compat MNG-5146 (WRONG_PARENT): bez flagi → FATAL mismatch; z personality → WARNING (demotion, nie unifikacja). Zero zmian w kodzie produkcyjnym.

Plik: ParentCycleDetectionTest.java (+~151 linii). Dodane importy: Constants, assertFalse. Istniejący test shade FATAL+hint pozostaje nietknięty (poza kanoniczną parą).

Nowe testy (nazwy komunikują „demotion, not unification”):

- testExplicitRelativePathGaMismatchIsFatalByDefault — FATAL na komunikacie parent.relativePath + ../pom.xml; assertFalse na WARNING dla tego mismatchu.
- testExplicitRelativePathGaMismatchDemotedToWarningWithMaven3Personality — System property Constants.MAVEN_MAVEN3_PERSONALITY=true, potem ApiRunner.createSession() (recreate; userProperties pod ApiRunner to anti-pattern), assert WARNING i brak FATAL mismatchu; cleanup w finally.

ParentCycleDetectionTest — fragment demotion (personality → WARNING)

```
String previous = System.getProperty(Constants.MAVEN_MAVEN3_PERSONALITY);
try {
    System.setProperty(Constants.MAVEN_MAVEN3_PERSONALITY, "true");
    // ApiRunner snapshots System properties into config at create time; recreate session.
    session = ApiRunner.createSession();
    modelBuilder = session.getService(ModelBuilder.class);
    // ... build request, collect problems ...
    assertTrue(hasWarningMismatch, "Expected mismatch demoted to WARNING...");
    assertFalse(hasFatalMismatch, "Personality must demote the relativePath GA mismatch...");
} finally {
    /* restore or clear System property */
}
```

Helper layoutu (SoT = MNG-5146): plik z GA wrong:wrong oraz child z parent mygroup:myparent i explicit <relativePath>../pom.xml</relativePath>:

writeWrongParentExplicitRelativePathLayout — kanoniczny układ POM

```
/* tempDir/pom.xml */
<groupId>wrong</groupId><artifactId>wrong</artifactId>...
/* tempDir/child/pom.xml */
<parent>
  <groupId>mygroup</groupId><artifactId>myparent</artifactId><version>1.0</version>
  <relativePath>../pom.xml</relativePath>
</parent>
```

Faza 3 — Paired verification gate (commit b666e22dc7 + epilogue b7a73fd9f6)

Cel: potwierdzić parę impl↔compat przy celowo różnych severity. Plan: zero nowego kodu Java.

- Automated: ParentCycleDetectionTest (7) oraz compat DefaultModelBuilderTest (3, w tym MNG-5146 WARNING).
- Review: ten sam kształt WRONG_PARENT; brak shared test-jar / wspólnego classpath resource; brak unifikacji severity.
- Opcjonalny smoke ITS MavenITmng8294ParentChecksTest — FATAL mismatch w dystrybucji; nie testuje demotion personality.
- Pliki: Progress w plan.md; change.md → status implemented.

Podsumowanie plików

Faza	Plik	Rodzaj zmiany
1	DefaultModelBuilder.java (impl)	+1 linia komentarza (logika bez zmian)
1	contract-twin-relativepath.md	nowy plik — kontrakt severity
2	ParentCycleDetectionTest.java	+2 testy + helper WRONG_PARENT + collectProblems
3	(brak kodu Java)	verify / review / Progress / status implemented

10. Jak to się ma do całego M4 / „realizing change”

W ścieżce lekcji / pracy M4 widać świadomy ciąg:

- Mapa terenu (repo-map) — gdzie jest system i co boli.
- Deep focus (model-builder-flow-analysis) — co konkretnie jest długiem na hot path.
- Ranking i plan (refactor-opportunities) — co warto robić najpierw i jak, bez scope creep.
- Realizacja — małe, odwracalne kroki (guard-first); K2 domknięte; potem ewentualnie K1/K3 jako follow-up.

To jest właśnie „realizing change” w sensie inżynierskim: nie wielki bang refaktoru, tylko zmiana uziemiona w dowodach, z jawnym „czego nie robimy” i z najmniejszym możliwym blast radiusem. Dzięki temu AI agent / contributor nie „ulepsza” Mavena w sposób, który psuje most migracyjny M3↔M4.

Follow-upy (świadomie później): extract stage’ów K1, characterization+extract K3, pomiar pokrycia, research ClassRealm, adopcja personality w ekosystemie. Osobno — gdyby kiedyś „naprawiać” severity twinów — to już redesign kontraktu domeny, nie struktura.

11. Podsumowanie dla kogoś spoza zespołu

Problem biznesowo-techniczny: Maven 4 musi być ostrzejszy niż Maven 3 w niektórych regułach (np. niespójny parent), ale ekosystem nadal potrzebuje mostu i trybu „osobowości M3". Różnica zachowań jest celowa — i mamy już tarczę testową + dokument kontraktu przed przypadkowym zatarciem.

Dlaczego ta zmiana (i status teraz): mapa wskazała strefę modelu jako #1 ryzyko; ranking mówił „najpierw K2"; plan SOUND; implementacja na fix/test-implementation domknęła guardy bez zmiany semantyki produktu.

Czego ta zmiana nie jest: nie jest przebudową architektury Mavena, nie jest unifikacją M3 i M4, nie jest „czyszczeniem 3450 LOC". Jest ubezpieczeniem przed fałszywą poprawką.

> Zdanie na plakat
>
> Zrobiliśmy refactor-opportunities, bo mapa i research wskazały najdelikatniejsze miejsce Maven 4 — a my domknęliśmy najmniejszy krok, który chroni celową różnicę M3↔M4, zanim ktokolwiek ruszy duży hub.

Źródła (w repozytorium)

- context/map/repo-map.md — mapa onboardingowa i strefy ryzyka
- context/changes/model-builder-flow-analysis/ — prior długu ścieżki ModelBuildera (+ research.md)
- context/changes/refactor-opportunities/change.md — intencja i etapy (status: implemented)
- context/changes/refactor-opportunities/research.md — ranking K2 → K1 → K3 + weryfikacja ast-grep
- context/changes/refactor-opportunities/plan.md (+ plan-brief.md) — zakres pełne K2; Progress faz 1–3 domknięty
- context/changes/refactor-opportunities/contract-twin-relativepath.md — notatka kontraktu severity
- context/changes/refactor-opportunities/reviews/plan-review.md — werdykt SOUND (po F1/F2)
- fix/test-implementation — 9d643a181c (p1), ebcd0c331b (p2), b666e22dc7 (p3), b7a73fd9f6 (epilogue)

12. Historia zmian

Krótki log tego dokumentu PDF — żeby było widać, jak narracja rosła od planu do zaimplementowanego K2 (bez kompresji wcześniejszych sekcji).

Data	Wersja	Co się zmieniło
2026-10-14	v1 — plan	Pierwsza notatka: mapa → prior → ranking K2→K1→K3 → plan pełne K2; status plan_reviewed (~9 stron).
2026-10-14	v2 — skrót (odrzucony)	Próba dopisania implementacji przez skompresowanie prozy — za krótka; nie jest wzorcem.
2026-10-14	v3 — implementacja	Pełna proza v1 + sekcja 9 (fazy 1–3, code_block); status implemented; branch fix/test-implementation.
2026-10-14	v4 — nawigacja	H1/H2 wyróżnione kolorem i rozmiarem; klikalny spis treści + bookmarks; ta sekcja Historii.

Źródło reguł generowania: .claude/prompts/m4/m4-realizing-change-reasons-pdf.md.

Wygenerowano lokalnie na potrzeby notatek M4 · plik: .claude/notes/m4/m4-realizing-change-reasons.pdf · layout v1 + H1/H2 kolor · klikalny TOC · historia · 2026-10-14